

The shopping agent, explained from scratch

This document assumes no knowledge of the project or of search / recommendation systems. It covers three things: **what the system does**, **how it is built**, and **how well it works** against two different tests.

Branch: `bm25_only` — the agent uses only classic keyword search, no AI language model. (Two experiments with a small embedding model are recorded in `docs/team/dense_rerank.md` and `docs/team/dense_route.md`; neither helped, so this branch does not carry them.)

1. The task

There is a **catalog** of 50,000 real clothing / shoe / jewellery products. One of them is a **hidden target**. A **simulated shopper** (a computer program, not a person) starts a chat: *"I'm looking for a belt, and it has to be leather."*

The agent must reply each turn with:

- a short natural-language message (e.g. a follow-up question),
- optionally one structured question ("ask about: colour"),
- a **ranked list of up to 10 products** — its best guesses at the target.

The shopper answers questions, revealing a bit more each turn, for up to 10 turns. The moment the target appears anywhere in the agent's list, the session ends and is scored.

How a session is scored

Three numbers, combined:

name	plain meaning
Hit Rate @10	fraction of sessions where the target showed up in the top 10 at all
MRR (mean reciprocal rank)	<i>how high</i> it was ranked. Rank 1 -> 1.0, rank 2 -> 0.5, rank 4 -> 0.25, not found -> 0. Rewards getting it to the very top.
Efficiency	how <i>few</i> turns it took. Finding it on turn 2 beats turn 8.

Final score = $0.50 \times \text{HitRate} + 0.30 \times \text{MRR} + 0.20 \times \text{Efficiency}$

A perfect agent scores 1.0. The organiser runs 200 public sessions you can see and 800 private ones you cannot.

2. How the agent is built

The agent is a **pipeline**: seven stages, each doing one job, passing its output to the next. Nothing here is machine-learned — it is all explicit rules and counting. It runs offline in about 17 milliseconds per turn and needs no internet and no paid services.

```
shopper's messages
|
---v--- S1 build a searchable index of the 50,000 products (once, at startup)
| index |
|-----|
---v--- S2 is this shopper "buying" (decisive) or "browsing" (vague)?
| router |
|-----|
---v--- S3 remember everything the shopper has said so far
| memory |
|-----|
---v--- S4 decide what to ask next
| policy |
|-----|
```

```

---v--- S5 pull ~300 candidate products out of the 50,000
|retrieve|
-----
---v--- S6 re-sort those 300 so the best guesses are on top
| rerank |
-----
---v--- S7 decide whether to show a list this turn, and how long
| timing |
-----
the reply

```

S1 — Index (src/index.py)

Think of a library card catalog. Every product's title, features, description, category and brand are loaded into a fast text database (SQLite's "FTS5"). When you give it words, it returns the products that contain those words, ranked by a formula called **BM25**.

BM25 in one paragraph. It scores a product against your search words by adding up, for each word: (a) how *rare* the word is across the whole catalog — "titanium" tells you far more than "the"; (b) how *often* the word appears in that product — but with strongly diminishing returns, so five mentions of "leather" is not five times better than one; (c) a penalty if the product's text is very long (a long description mentioning your word in passing is weaker evidence than a short one that's all about it). Different parts of the listing count differently — a word in the **title** counts about 32× as much as the same word buried in the marketing blurb.

BM25 is pure word-matching. It has no idea that "leather" and "cowhide" mean the same thing.

S2 — Router (src/router.py)

Looks at the shopper's opening line and decides: **buying** (they know what they want — "*a key requirement is: waterproof*") or **browsing** (they're exploring — "*I'm still just looking*"). If it's unsure, it assumes browsing, because wrongly treating a vague shopper as decisive locks in guesses they never made.

The track is re-checked every turn — a browser who starts naming specific requirements is promoted to "buying".

S3 — Memory (src/state.py)

The naive version of this agent answered every turn using only the *latest* message and forgot everything else. That was its single biggest weakness. This stage keeps **every constraint the shopper has ever mentioned** and feeds the accumulated list to search. If the shopper later reverses themselves ("actually, forget leather — I want canvas"), the old preference is *down-weighted*, not deleted (erasing it turned out to lose useful signal).

S4 — Question policy (src/policy.py)

Decides the one structured question to ask next. The default ("is there anything else that matters?") is deliberately broad — the simulated shopper will happily volunteer their next requirement, so a broad question extracts the most per turn. On the **browsing** track the agent instead asks the question that would best split the current candidate list ("you've got leather and canvas here — any preference?"), because a vague browser does *not* volunteer things unprompted.

S5 — Retrieval (src/retrieval.py)

Runs the keyword search three ways and blends the results:

- **terms** — search using every word the shopper has said (maximises the chance the target is somewhere in the pool),
- **anchor** — search using only the opening line (keeps the results anchored to the original product category),
- **focused** — after a mind-change, search using only what was said *after* it.

Each returns up to 300 products; they're merged by **reciprocal-rank fusion** (a product ranked highly by any one of the three floats to the top of the combined list). Final output: the top 300.

Why 300 out of 50,000? Because the simulated shopper's requirements are copied word-for-word from the target's own product page, so the target almost always contains every search word and lands comfortably inside the top 300. Retrieval is a nearly-solved problem *for this simulator* — the hard part is ranking.

S6 — Rerank (`src/rerank.py`)

Takes those 300 and re-sorts them. BM25 got the target *into* the pool but often only at position ~200 (its requirements are generic phrases hundreds of belts share). This stage rescues it, by scoring each candidate on:

- **exact phrase coverage** — does the product's text literally contain "buckle closure", "stainless steel band"? This is the strongest signal, because the shopper's phrases are quoted straight from the target's page.
- **popularity** — a real, reviewed, well-documented product is more likely to be the target than a bare listing with the same words.
- **category / attribute agreement** — does its category path and its material / colour line up with what the shopper said?
- **contradiction penalty** — if the shopper said "grey" and this product's colour is definitely black, push it down.

S7 — Timing (`starter/agent.py`)

Showing a list ends the session the instant the target appears in it — freezing whatever rank it held *then*. So the agent holds back until turn 3, shows a short list of 4 first (banking a good early rank only if it's confident), then widens to 10. It also runs an **elimination scan**: any product it already showed and didn't score a hit on is a confirmed non-target, so it drops those and shows the next batch down.

The dual-track layer

By default the buying/browsing track from S2 drives S4 (which question), S6 (which rerank weights) and S7 (when to recommend). `use_router=False` turns all of that off and runs one flat pipeline. On the official test the flat pipeline actually scores slightly higher (see §4) — the routing is kept because it pays off on the harder test in §3, not on the easy one.

3. The two evaluators

The original evaluator (`evaluator/local_evaluator.py`) — *frozen, official*

This is the scorer the competition uses. Its simulated shopper is **completely cooperative**:

- it always answers when asked,
- every requirement it states is **copied verbatim from the target's own product page** ("100% Leather", "Buckle closure"),
- its "change of mind" swaps one target-derived value for another — never a real retraction.

This is a fair, exact test of *retrieval and ranking*. But it cannot test whether the agent copes with a shopper who speaks differently from the catalog, or one who won't volunteer information, because this shopper never does either. You are not allowed to modify this file, and its score is the only one that counts for the competition.

The new evaluator (`tools/stress_harness.py`) — *a robustness probe*

Same scoring maths, same session loop, same *unmodified* agent — but the simulated shopper can be made harder in composable ways:

stressor	what the shopper does
paraphrase:light	states the same requirements but rewords the sentence around them ("For that, what matters is: X" -> "It should be X.")
paraphrase:medium	rewords the requirement itself ("colour: blue" -> "in blue"; "100% Cotton" -> "all cotton")
paraphrase:heavy	medium, plus swapping words for synonyms ("leather" -> "cowhide", "waterproof" -> "water-repellent") and shuffling the phrasing
browse-gated	a vague browser who reveals a requirement only when asked a pointed question about that exact attribute — never on a broad "anything else?"
decoy	genuinely changes its mind: the requirement it drops is a real value the target does not have

These compose: paraphrase:heavy + browse-gated is a vague browser who also rewords everything — the closest thing to a feared "difficult" private shopper.

The harness also reports, for every failed session, **whether retrieval or ranking broke**: did the target never make it into the 300-candidate pool (a retrieval failure), or was it in the pool the whole time but never shown (a ranking failure)?

A --verify mode confirms that with no stressors applied, the harness reproduces the official evaluator's score to nine decimal places — so the deltas it reports are real, not artefacts.

These numbers are a robustness estimate, not a competition score. The stressors are hand-written; they are informed speculation about the 800 private sessions, used to decide what to make more robust and to write up in the report.

4. Results

On the original (official) evaluator

Two versions of the agent: **flat** (use_router=False, one pipeline) and **dual-track** (use_router=True, the default on this branch).

test set	sessions	flat agent	dual-track agent
Public	200	0.9305	0.9177
Public — Hit Rate @10	200	200 / 200	200 / 200
Adversarial ("hard")	96	0.8020	0.7994
Dev split	120	0.9418	0.9268
Holdout split	80	0.9136	0.9041

Public score by shopper type (dual-track agent): buying 0.90 MRR, browsing 0.86, change-of-mind 0.84, "no preference" 0.81. Hit Rate is a perfect 1.00 for all four — the target is *always* found; the only variation is how high.

Reading it. The agent finds the hidden target in every one of the 200 public sessions and ranks it near the top. The flat agent is marginally ahead (0.9305 vs 0.9177) because this cooperative shopper rewards nothing but broad questions and early lists — exactly what the flat pipeline does. The routing costs ~1.3 points here and its benefit is invisible to this test, which is the whole reason the next section exists.

On the new evaluator (stress harness), dual-track agent

shopper	Hit Rate @10	MRR	score	Δ vs cooperative	constraint words the agent actually saw
cooperative (= official)	1.00	0.87	0.9177	—	79%
paraphrase:light	0.99	0.79	0.8869	-0.031	81%
paraphrase:medium	1.00	0.77	0.8822	-0.036	71%
paraphrase:heavy	0.97	0.75	0.8558	-0.062	66%
browse-gated (vague browser)	0.98	0.79	0.8775	-0.040	68%
paraphrase:medium + browse-gated	0.94	0.73	0.8251	-0.093	62%
paraphrase:heavy + browse-gated	0.93	0.67	0.8007	-0.117	57%
decoy (genuine change of mind)	1.00	0.88	0.9194	+0.002	80%

Reading it.

- ****Paraphrasing hurts, and it hurts *ranking*, not *finding*.** Even under the heaviest rewording the target is still shown in 97% of sessions (Hit Rate barely moves), but its average rank drops from ~1st to ~3rd–4th (MRR 0.87 $\rightarrow$ 0.75). The reason is S6: its strongest signal is *exact phrase* matching, and "cowhide" is not "leather", so that signal goes silent and the target competes on weaker evidence.
- **The realistic worst case is ~0.80.** A vague browser who also rewords everything. If the private shopper behaves like this, the real score is around 0.80, not 0.93.
- **A genuine change of mind is handled fine** (+0.002). The down-weighting machinery in S3, which does nothing on the official test, earns its keep here.
- **The dual-track routing pays off here.** On the same browse-gated shopper, the flat agent scores **0.73** and the dual-track agent **0.88** — a +0.15 swing. The flat agent keeps asking "anything else?", the browser reveals nothing, and the target is never found in 41% of browsing sessions. The dual-track agent switches to pointed questions and recovers it. And mis-routing is very lop-sided: treating a browser as a buyer costs about 0.66 MRR; treating a buyer as a browser costs about 0.07.
- **Where retrieval finally does break:** only the hardest corner — a browser + heavy paraphrase — pushes ~12% of browsing targets out of the 300-pool entirely. Everywhere else, retrieval holds and ranking is the story.

What was tried and did not work

A small embedding model (bge-small) was added to bring "meaning-based" matching to the reranker and to retrieval, on the theory that it would fix the paraphrase problem. It did not: **-0.003** on the full paraphrase test, a real but tiny **+0.02** only on the ~10% of sessions where every requirement is both generic and reworded, and it recovered **none** of the lost-from-pool targets. The reason is that retrieval has already narrowed the field to one product category, so every candidate's embedding says "belt" loudly and the reworded *attributes* are lost in the noise. Full write-ups: docs/team/dense_rerank.md, docs/team/dense_route.md. This branch removes that code.

5. One-line summary

The agent is a fast, offline, seven-stage keyword-search pipeline that finds the hidden product in **100% of the official test sessions** and ranks it near the top (score **0.93** flat / **0.92** with buying-vs-browsing routing). A separate harness shows that if the private shopper paraphrases and stays vague, the score is more like **0.80**, with the loss concentrated in ranking because the reranker matches exact phrases — and that the buying-vs-browsing routing,

which slightly costs on the easy test, is worth **+0.15** on that harder shopper.